

COMP207 – Main assignment:

The supermarket database

There are 100 points you can get in this assignment. Recall that the assignment contributes 25% to your overall grade in the course, meaning each point corresponds to 0.25% of your final grade.

The assignment is to create a database for a supermarket: The initial task will focus on building the database (i.e., tables and so on), then adding in some data, while the latter tasks focus on creating useful queries. The number of points for each question is marked on them.

For questions 1-2, the number of points you get for your submission is public, i.e. it is given to you by CodeGrade a few minutes after you submit.

For questions 3-8, the points are given for two sub-tasks:

- **Public data:** For each question, 2 points are given for getting the right output on the public test data described in an additional file (INSERT Statements for public test data (25-26).sql) and you will be told how many of these your current submission gets you a few minutes after you submit. The question will specify what this right output is, and you will also be able to see it when you upload your solution to CodeGrade (since you can upload any number of times before the deadline, I would suggest using it to test your solution against the test data)
- **Hidden data:** The remaining points are given for getting the right output on other data sets. The latter sets are kept hidden to avoid you hardcoding the right output in the queries. The sets will follow the same form as the test data though and unless you hardcode your solution for the test data, it is very likely that if you get points for one part you will get points for the other. Multiple sets are used to minimize the chance that a single minor mistake cost many points.

Please ask questions on the discussion board instead of over emails. For most of you, most questions you are likely to come up with have likely already been answered.

Each question from 3-8 requires you to create a view with a specified (in the question) name. You may use as many views as you wish to solve these questions, but there should be one with the specified name, which is the one that is getting checked for having the right output. E.g., you could have **view1ForQuestion3**, **view2ForQuestion3** and **view3ForQuestion3**, if you feel it would help to do question 3 – most other names are fine too (but it might be helpful for you if the names were descriptive)!

LLM-style chatbots: You are not allowed to use LLM-style chatbots, such as ChatGPT.

Group work: While you might discuss with your friends/colleagues, you must hand in a solution only you worked on, and plagiarism checks will be run.

How to get error messages from CodeGrade

The location of the error messages your submission gets is perhaps not obvious so please check this!

After having run your submission, you will get an output like this (without a **red** circle):

This is a preliminary result. Hidden steps have not been run yet and the AutoTest configuration may change. Therefore, the final result may differ.

> Setup

Categories

Hardcode testing

No	Summary
1	Checks whether you hard-coded which database to use.
> 1.1	We put your database (specifically the customers table) into a database called test_db and see if we can find the information from there. If that works, great, otherwise, you forced it into some specific database. Note, while this is just for the customers table, you should not force any of the tables or views.

Question 1: Create Tables

No	Summary
1	Question 1: Create Tables
> 1.1	Check whether the Customers table is constructed correctly.
> 1.2	Check whether the Employees table is constructed correctly.
> 1.3	Check whether the Deals table is constructed correctly.

Click on the > in the **red** circle and scroll down some. You will see something like this:

Per-student setup script

```
bash $FIXTURES/student_setup.sh
```

The bash command that will run your per-student setup script, which will be run for each student. See [our documentation](#) for more information. The command cannot take longer than 15 minutes, after which it will be killed.

stdout

stderr

No output.

That stderr will contain your error message.

Deadline and feedback

The deadline for the assignment is **Thursday the 13th of November at 17:00**. General feedback for the assignment will be given on Tuesday the 2nd of December.

If you have specific concerns about your grade or similar for the assignment, then, after the general feedback has been released, I will answer questions about your solution and grade over email.

Lateness penalties

The standard University of Liverpool policy regarding late submissions applies to this assessment.

- Every student may submit up to 7 calendar days after the deadline, without penalty.
- Students who have a Student Support Information Sheet (SSIS) that includes the "coursework deadlines extensions allowed" adjustment can make use of a further 7-day extension period (total of 14 calendar days after the deadline).
- Any attempt to submit more than 7 days (or 14 days with an adjustment) after the deadline will be classed as non-submission and will receive a mark of zero.

Format

The assignment should be done in .sql format (i.e., the output format from MySQL's workbench) – it is really just a basic text file with the SQL commands written in it and you could do it by writing the file directly in a basic text editor if you wish (Notepad in Windows or TextEdit on Mac – if you select Make Plain Text in Format).

The name of the file should be supermarket.sql: You can hand in precisely 1 file, and it must have precisely that name (you can submit as many times as you wish until the deadline, but only the most recent version counts).

And each line should contain only the following:

1. CREATE TABLE statements for question 1 (7 in total)
2. INSERT statements for question 2 (depending on how you write them, but likely up to 7)
3. CREATE VIEW statements for questions 3-8 (the number of views depends on how you solve the questions and how many you solve, if not all). Note that you may use any positive number of views to solve each question, but each question's specified view should have the properties requested.
4. SQL comments, i.e. the part of lines after "-- ", i.e. double - followed by space. You do not need to make any but may do so if you wish.

Do not include CREATE DATABASE and USE statements. They will make the tests on the hidden data not work (technically, I create multiple databases based on your construction, one with the public data given in the additional file (INSERT Statements for public test data (25-26).sql) and a number with the hidden data. If you use CREATE DATABASE or USE statements, in essence only 1 is made and it will be marked as if you did not do the other ones). I have written tests that test for this so if you do not do something really complex to avoid these, you should at least be warned about it.

Make sure that you can run the full file through MySQL when using the Supermarket database (starting with an empty Supermarket database) and after having done so, the Supermarket database should contain the tables and views required from the questions you solved (and perhaps some more views if you feel it would be convenient). This means that **you should remove any statement that**

causes errors before handing in the assignment because MySQL stops when it encounters an error (meaning that the last statements are not executed)! If you do not, you risk getting a far lower grade than otherwise (because the part of your hand-in after the first error will not be graded).

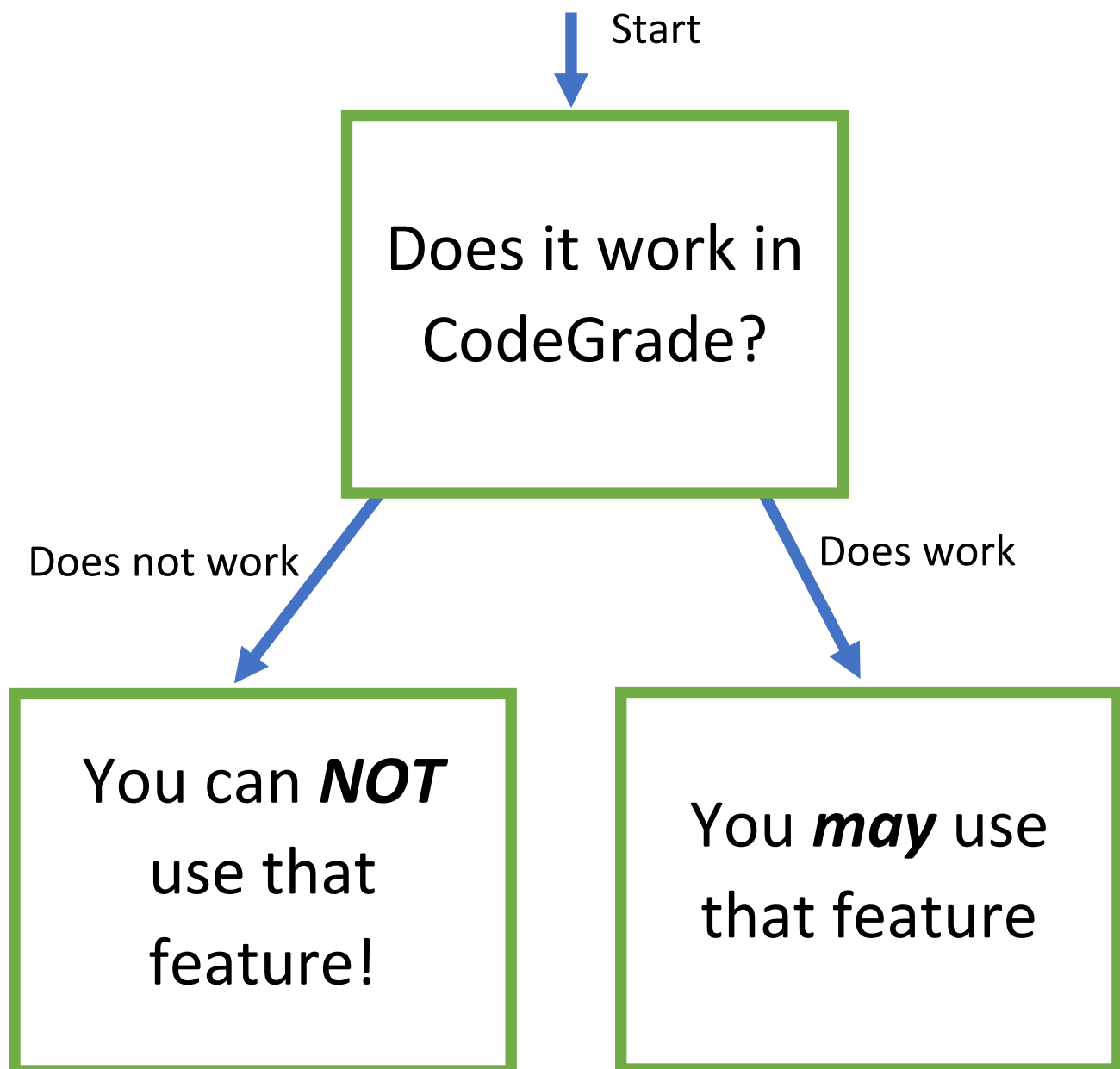
You can submit any number of times before the deadline: We are using CodeGrade for checking these things and whenever you submit, you will see whether your file works for the public data set. I suggest using it...

Do *not* do the following!

Any of the following should **not** be done:

- End by removing the database (i.e. `DROP DATABASE Supermarket;` or similar). It would be the same as handing in an empty file. It will be very easy to see on CodeGrade since everything will stop working.
- Create comments like `"-----"`. MySQL Workbench will accept it, but the command line version of MySQL does not, which is what is used to check your file... Just insert an extra space after the second `-`.
- Use `PARTITION OVER`, `RANK OVER` or other new commands. It was not taught in the course and is specific to newer versions of MySQL (like the one you would install on your own laptop). Unfortunately, CodeGrade is using an old version of Ubuntu (or at least did last year), where the newest versions of MySQL do not work. Therefore, it will not work when we grade you and you will fail that (and later questions – MySQL will report an error on that line and will not run the rest of your file).
- It is very unlikely that you would want to remove any views after you have made them. Unless you have a very good reason, do NOT remove them. Doing so anyway typically leads to strange error messages.

Flow diagram to help you decide if you may use a specific feature that was not taught in the course



Question 1 – (Easy) (worth 16 points – 2 points for each table and an additional 2 as part of a check that you did not use USE or CREATE DATABASE. You will be told by CodeGrade directly soon after submission whether you get these points or not for the current submission)

Make the set of tables that match the following set of schemas.

- **Customers**(first_name,last_name,birthday,c_id)
- **Employees**(first_name,last_name,birthday,e_id)
- **Deals**(value,d_id)
- **TypesInDeals**(d_id*, type)
- **Stock**(amount,type*,name)
- **Transactions**(date,challenge,c_id*, e_id*, t_id)
- **ItemsInTransactions**(name*,cost,t_id*)

Each underlined attribute should be the primary key for the table (it happens to be the last attribute in the first 5 tables) and each attribute with * should have a foreign key to the table with a primary key of just that name, e.g. if the tables were R(a,b) and S(b*,c), b in R and c in S should be the primary keys and b in S should reference b in R as a foreign key. More directly, d_id in TypesInDeals should reference d_id in Deals, type in Stock should reference type in TypesInDeals, c_id in Transactions should reference c_id in Customers, e_id in Transactions should reference e_id in Employees, t_id* in ItemsInTransactions should reference t_id in Transactions and name in ItemsInTransactions should reference name in Stock.

Only use data types in the following list: INT, VARCHAR(20), DATE and BOOL. Instead of specifying the datatypes explicitly, ensure that the public test data defined in the additional file (INSERT Statements for public test data (25-26).sql) gets inserted correctly. Mostly, I imagine you would be able to guess what the data types should be: The exceptions are

- that we use cost and values in pennies (and thus INT), because floating point numbers can give rise to rounding errors, depending on how you use them (INT avoids any such issues)
- type is a VARCHAR(20)
- and challenge is (the only) BOOL

NOTE about help for this question: If you struggle with this question, do ask questions and we (the TAs and I) will try to *guide* you to a solution (but do try first). You can also ask on the discussion board.

Question 2 – (Easy) worth 14 points (you will be told directly by CodeGrade soon after submission whether you get these 14 points for the current submission)

Insert the following information into the database (using INSERT statements):

- A customer with the name of Ben Thompson, customer id 6 and having his birthday the 21st of July 1992.
- An employee with the name of Rita Davies, employee id 5 and having her birthday the 7th of October 2000.
- A transaction between Ben and Rita, on the 7th of September 2025, with no challenge and transaction id 18.
- There are 7 newspapers in stock (i.e. with name Newspaper) and 4 pens (i.e. with name Pen) both of no type (i.e. the type should be null – null indicates that they are not in a deal).
- Two items were sold in the transaction between Rita and Ben, a newspaper with a cost of 149 and a pen of cost 99.

NOTE about help for this question: If you struggle with this question, do ask questions and we (the TAs and I) will try to *guide* you to a solution (but do try first). You can also ask on the discussion board.

Question 3 – (Easy) (worth 10 points – 2 points for getting the right output on the public test data and another 8 for the hidden data – see the first page for more detail!)

Find the total number of transactions sold by David Smith, who has employee id 4, in August 2025.

More precisely, create a view, **August2025SalesByDavid**, with one column, `number_of_sales`, that outputs the number of transactions sold by employee number 4 (who is named David Smith, but you do not need to verify that) in August 2025. You may assume that some sales were made that month by employee 4 (it turns out to be somewhat more complex if you do not assume so).

Note that in the public test data, he did 6 transactions in that period, specifically, transactions with id 7, 8, 10, 11, 15 and 16. The remaining transactions were either done by some other employee or earlier or later than August 2025.

Hint: Recall that `COUNT(*)` will count the number of rows in a query, so you just need to make a query that finds the transactions sold by David in August 2025 and then use that.

NOTE about help for this question: If you struggle with this question, do ask questions and we (the TAs and I) will try to *guide* you to a solution (but do try first). You can also ask on the discussion board.

The view should be called **August2025SalesByDavid** and be such that the output of

```
SELECT * FROM August2025SalesByDavid;
```

when run on the Supermarket database (after inserting the public test data given in the additional file) should be:

<code>number_of_sales</code>
6

Question 4 – (Medium) (worth 10 points – 2 points for getting the right output on the test data and another 8 for the hidden data – see the first page for more detail!)

While in the UK, you need to be at least 18 years old to buy alcohol, many retailers, including the supermarket this database is for, want to challenge everybody who looks under 25. Each transaction in the database has a challenge Boolean, stating whether the customer was challenged.

We want to find, for each transaction that included Alcohol (a type), whether the customer was challenged and whether they were above 25. To simplify that, we will use that people who are at or above 9132 days old are certainly above 25 (with anyone below 9130 being less than 25 - due to leap years, there is not an exact number).

Given two dates (or in this case likely a date and a datetime), X and Y, you can determine the number of days between them using DATEDIFF(X,Y). To make this question simpler, you are meant to determine whether a customer is at least 9132 days old when they buy alcohol.

More precisely, for each transaction that involves Alcohol (you need to look up the items for a given Transaction in ItemsInTransactions and their type in Stock), display the transaction id, t_id, whether the customer was challenged, challenge, and whether they were 9132 days old, above_25.

In the public test data, Alice Brown is below 25 each time she buys alcohol (transactions 2, 6, 7, 14 and 17), Clara Nguyen is below twice and then above once (transactions 9, 11 below and 15 above) and Ben Thompson is above 25 each time (transactions 12 and 16).

NOTE about help for this question: If you struggle with this question, do ask questions and we (the TAs and I) will try to give you some *hints* towards a solution (but do try first). You can also ask on the discussion board.

The view should be called **Above25** and be such that the output of

```
SELECT * FROM Above25 ORDER BY t_id;
```

when run on the Supermarket database (after inserting the public test data given in the additional file) should be:

t_id	challenge	above_25
2	1	0
6	1	0
7	0	0
9	1	0
11	1	0
12	1	1
14	0	0
15	1	1
16	0	1
17	1	0

Question 5 (Medium) (worth 10 points – 2 points for getting the right output on the test data and another 8 for the hidden data – see the first page for more detail!)

In this question, you are meant to find how much stock is left *after* all the sales. Note that the Stock table contains how much was in stock *before* the sales.

To be more precise, you should create a view, **StockLeft**, that has two columns, name, and stock_left, that for each name of an item says how much is left in stock of that item. For simplicity, you may assume that there were some of the items in stock and some got sold (you can also assume that there is a non-negative amount left, but it seems unlikely that your query will use that).

Examples from the public data set: As an example, we sold toilet paper 3 times (in transactions 6, 9 and 10) and we started with 3 in stock, so we have 0 left. We sold wine 7 times (in transactions 2, 6, 7, 9, 12, 14 and 17) and started with 8 in stock, so we have 1 left.

NOTE about help for this question: If you struggle with this question, do ask questions and we (the TAs and I) will try to give you some *hints* towards a solution (but do try first). You can also ask on the discussion board.

Hint: You should likely use GROUP BY to solve this problem. An easy solution could be to create a view that contains how much got sold (this would be the one that used GROUP BY and likely COUNT) and then create StockLeft from that by subtracting how much got sold from how much was sold. That said, you can do it in one view, but it is a bit uglier.

The view should be called **StockLeft** and be such that the output of

```
SELECT * FROM StockLeft ORDER BY name;
```

when run on the Supermarket database (after inserting the public test data given in the additional file and data from question 2) should be – the right columns are the continuation of the left columns:

name	stock_left
Apple	4
Banana	13
Beer	14
Butter	2
Cereal	2
Chicken breast	1
Chocolate	7
Coffee	10
Eggs	11
Fried rice	2
Milk	3
Newspaper	6
Orange juice	3
Pasta salad	3
Pen	3
Pie	10
Pizza	4
Potato	4

name	stock_left
Sandwich	2
Soap	3
Soda	2
Spaghetti	6
Steak	3
Sushi	2
Tea	6
Toilet paper	0
Wine	1

Question 6 – (Medium) (worth 15 points – 2 points for getting the right output on the test data and another 13 for the hidden data – see the first page for more detail!)

In August 2025, the supermarket was running a promotion, where some lucky customers would win a prize, with each transaction being awarded a ticket into the prize draw.

However, employees are not eligible for these prizes (we assume that a customer is an employee if the customer and employee have the same name and birthday). To make up for it, the supermarket additionally gave a prize to a customer who is also an employee based on a similar draw for them.

To facilitate this prize draw, you are meant to create a view, **PrizeDraw**, that outputs how many transactions each customer did in August 2025 and whether they are an employee as well. Specifically, the view **PrizeDraw**, should have three columns, `c_id`, for the customer id, `number_of_transactions`, for how many transactions they did in August 2025, and `is_employee` for whether they have the same name and birthday as some employee (be aware that the customer should match a single employee, and e.g. having an employee with the same name and another employee with the same birthday would not be enough – e.g. in the public test data, customer 3 match employee 1, customer 4 match customer 4, but customer 1 does not match anyone – while there is an employee with the same name and another with the same birthday that is not enough).

The output should be sorted by `is_employee` and then `number_of_transactions` and then `c_id`.

Examples from the public data set: First comes the non-employees (because we are sorting them by `is_employee` increasingly): Customer 1 made 3 transactions in the period (transactions 6,7 and 14), customer 2 made 3 transactions in the period as well (transactions 8, 12 and 16). Since neither is an employee, customer 1 is displayed first among those two. Customer 5 did not make any transactions in the period and is thus not displayed. Next comes the employees: Customer 4 is an employee (employee 4 to be precise), did 1 transaction in the period (transaction 13) and is displayed first. Customer 3 is also an employee (employee 1 to be precise) and did 4 transactions in the period (transactions 9, 10, 11 and 15) and is thus displayed last.

Hint: You likely need to use `GROUP BY` and `ORDER BY` for this question. From the lectures/videos, it is likely most straightforward to use `UNION`, but it is as such not necessary. If you do use `UNION`, it is often easiest to first do the `UNION` in one view and then sort it in another.

NOTE about help for this question: If you struggle with this question, do ask questions and we (the TAs and I) will try to give you some *hints* towards a solution (but do try first). You can also ask on the discussion board.

The view you create should be called **PrizeDraw** and it should be such that

```
SELECT * FROM PrizeDraw;
```

when run on the Supermarket database (after inserting the public test data given in the additional file) should be:

c_id	number_of_transactions	is_employee
1	3	0
2	3	0
4	1	1
3	4	1

Question 7 – (Hard)

(worth 10 points – 2 points for getting the right output on the test data and another 8 for the hidden data – see the first page for more detail!)

Over the next two questions, we will look at how much the customer should pay for their transactions, but we want to support deals (ala meal deals in Tesco).

Without deals, finding how much the customers should pay for a transaction would be easy: You would simply sum up the cost of each item in the transaction (the prices are in **ItemInTransactions**, because the prices change over time and depends on what the item is exactly – the name is just a short description).

However, we also want to support deals: Each deal (in **Deal**) has a value and is associated with some item types (in **TypesInDeals**). If you buy 1 of each type in the deal (meaning the type of the item, in **Stock**, matches the type in **Deal**), instead of costing the sum of the items cost, it cost the value given in **Deal**. Additionally, if the customer buys e.g. 3 of each type in the deal, they pay the value 3 times. Note that, as Type is the PRIMARY KEY for **TypesInDeals**, each (not-null) type is in exactly 1 deal – it would be NP-complete to solve the problem if type could be in any number of deals (but only counting towards 1).

The tricky part is what to do, e.g., if there is a deal with two items and the customer buys 2 of the first type in a deal and 4 of the second. The way the supermarket has decided to handle the example is that the customer pays for the deal twice and the two cheapest items of the second type (the cheapest because otherwise, the customer would just split up their transaction in 2 with the two cheapest items of the second type in one transaction and the remaining items in the other. Having the customer do so would slow down how fast they end up paying, which costs the supermarket money).

What you need to do in this question is to create a view, **NumberedItemsInTransactions**, that for each transaction, t_id, and each (not-null) type of item in that transaction, type, and each item of that type, name, how many items of that type in the transaction were at least as cheap as this item, cheapest (you may assume that the prices of different **ItemsInTransactions** of the same type in the same transaction are different. This makes this somewhat less realistic, but also easier) and what this item cost.

Examples from the public data set: Let's look at a few examples to make it clearer.

Transaction 1 consists of Milk, Eggs and Butter. As only Milk is a part of a deal (i.e. of not-null type), for transaction 1 we end up with:

t_id	type	Name	Count
1	Drink	Milk	1

Transaction 3 consists of Sandwich, Pasta Salad, Chocolate, Banana, Apple, Soda, Orange juice, Coffee and Tea (which are all of not-null types – they are all types in Deal 2) and for transaction 3 we end up with (due to their relative prices):

t_id	type	Name	cheapest
3	Drink	Soda	1
3	Drink	Orange juice	2
3	Drink	Tea	3
3	Drink	Coffee	4
3	Lunch	Sandwich	1
3	Lunch	Pasta salad	2
3	Sweet	Banana	1
3	Sweet	Chocolate	2
3	Sweet	Apple	3

Transaction 6 consists of Beer, Wine, Steak, Potato and Toilet paper, of which all but Toilet paper is of not-null type and for transaction 6 we end up with (due to their relative prices):

t_id	type	Name	cheapest
6	Alcohol	Beer	1
6	Alcohol	Wine	2
6	Main	Steak	1
6	Side dish	Potato	1

Transaction 9 is the same as transaction 6, but without Potato. We end up with

t_id	type	Name	cheapest
9	Alcohol	Beer	1
9	Alcohol	Wine	2
9	Main	Steak	1

Hint: You might want to first create a view that joins **ItemsInTransactions** and **Stock** together (to get type and name with cost). That view in turn should likely be in your final view twice in the FROM statement, to find, for each item, how many of the same type and in the same transaction were at least as cheap as it.

NOTE about help for this question: We will offer *clarification* of what you are meant to do in this question and the next question (i.e. of the aimed for output), if you are unsure, either in person or on the discussion board, but since these questions are meant to take your grade from 75 to 100, you are meant to do them yourself and we will *not offer hints* (besides the above).

The view you create should be called **NumberedItemsInTransactions** and it should be such that

SELECT * FROM NumberedItemsInTransactions ORDER BY t_id, type, cheapest;

when run on the Supermarket database (after inserting the public test data given in the additional file and the data from question 2) should be (the left columns continue in the right columns):

t_id	type	name	cheapest
1	Drink	Milk	1
2	Alcohol	Wine	1
2	Main	Chicken breast	1
2	Side dish	Spaghetti	1
3	Drink	Soda	1
3	Drink	Orange juice	2
3	Drink	Tea	3
3	Drink	Coffee	4
3	Lunch	Sandwich	1
3	Lunch	Pasta salad	2
3	Sweet	Banana	1
3	Sweet	Chocolate	2
3	Sweet	Apple	3
4	Main	Chicken breast	1
4	Main	Steak	2
4	Side dish	Spaghetti	1
5	Drink	Tea	1
5	Main	Pie	1
5	Sweet	Apple	1
6	Alcohol	Beer	1
6	Alcohol	Wine	2
6	Main	Steak	1
6	Side dish	Potato	1
7	Alcohol	Wine	1
7	Lunch	Pasta salad	1
8	Drink	Soda	1
8	Main	Pizza	1
8	Sweet	Chocolate	1
9	Alcohol	Beer	1
9	Alcohol	Wine	2
9	Main	Steak	1
11	Alcohol	Beer	1
11	Lunch	Sandwich	1
12	Alcohol	Beer	1
12	Alcohol	Wine	2
12	Drink	Tea	1
12	Lunch	Pasta salad	1

t_id	type	name	cheapest
12	Main	Chicken breast	1
12	Side dish	Fried rice	1
12	Sweet	Apple	1
13	Drink	Milk	1
13	Sweet	Banana	1
14	Alcohol	Wine	1
14	Lunch	Sushi	1
15	Alcohol	Beer	1
15	Drink	Coffee	1
15	Sweet	Apple	1
16	Alcohol	Beer	1
16	Lunch	Pasta salad	1
17	Alcohol	Wine	1
17	Main	Steak	1
17	Side dish	Potato	1

Question 8 – (Hard)

(worth 15 points – 2 points for getting the right output on the test data and another 8 for the hidden data – see the first page for more detail!)

Note: This question is building on top of 7. It seems unlikely that you can solve it without having solved 7 first.

As mentioned in 7, we are trying to find, for each transaction, what the customer should pay for that transaction, supporting the meal-deal style deals in **Deals**.

In this question, you are meant to create a view, **CostOfTransactions**, that for each transaction, gives its cost. More precisely, it has 2 columns, `t_id`, and the associated cost (in a column called `cost`). It will likely use **NumberedItemsInTransactions** from question 7 somehow.

Some examples from the public data set: Transaction 1 did not fulfill any deals (the Milk is a Drink, but you need to buy a Lunch and a Sweet as well to get the second deal), so the price is just the sum of the three items (from **ItemsInTransactions**): $120+250+230=600$.

Transaction 3 fulfilled 2 deals (both the same, namely deal 2). More precisely, the transaction has 4 drinks, 3 sweets and 2 lunches. The price is thus $300*2+100+100+240=1040$ – $2*300$ for 2 deals, each of cost 300, 100 is for the cheapest sweet, the banana, and $100+240$ are for the two cheapest drinks, the soda and the orange juice.

Transaction 6 fulfilled the first deal once. More precisely, the transaction has 2 Alcohols, 1 Main and 1 Side dish (and additionally Toilet paper which is of not-null type). The cost is thus $700+300+380=1380$ – the 700 is for the deal, the 300 for the cheapest alcohol, the beer, and the 380 is for the toilet paper.

Transaction 9 fulfilled no deals. More precisely, the transaction has 2 Alcohols and 1 Main (and additionally Toilet paper which is of not-null type). Note that there is no Side dish. The cost is thus $300+500+790+380=1790$ (if the customer had brought a potato like in 6, they would have paid quite a bit less as you can see).

Hint: One way to solve this problem is as follows:

1. Find the number of times each transaction satisfies each deal (based on **NumberedItemsInTransactions**).
2. Find the number of items of each type left after having done the deals (e.g. in transaction 1, there is a Drink left after having done the 0 deals, in transaction 3 there are 2 drinks and 1 sweet left after having done the 2 deals) – you should likely use your solution from step 1.
3. For each transaction, add up costs of the cheapest items of each non-null type (using **NumberedItemsInTransactions**) upto the amount left (from step 2)
4. For each transaction, add up the cost of items of null types
5. For each transaction, multiply the value of the deals (in **Deals**) with how many times the transaction satisfied that deal (from step 1) and sum up over the different deals
6. Add up the numbers from step 3, 4 and 5 to get the result

A lot of the complications in solving these steps, in particular for solving step 1, is that a number of things are present 0 times. That means that they won't show up in the output. E.g. if you try to find how many of each type were sold in each transaction, you will, at least in the more straightforward ways, find that in the public data set in transaction 1, 1 drink was sold, but not directly 0 lunches – it will simply not display lunches. You can add in the missing 0s by e.g. using UNION and iterating over each transaction id and type and then either add in exactly the missing 0s by using NOT (EXISTS ...) or adding in 0s everywhere and taking the maximum number for each combination of a transaction id and a type.

NOTE about help for this question: We will offer *clarification* of what you are meant to do in this question and the next question (i.e. of the aimed for output), if you are unsure, either in person or on the discussion board, but since these questions are meant to take your grade from 75 to 100, you are meant to do them yourself and we will *not offer hints* (besides the hint in question 7 and the above hints).

The view you create should be called **CostOfTransactions** and it should be such that

```
SELECT * FROM CostOfTransactions ORDER BY t_id;
```

when run on the Supermarket database (after inserting the public test data given in the additional file and the data from question 2) should be:

t_id	cost
1	600
2	700
3	1040
4	1290
5	730
6	1380
7	1370
8	820
9	1970
10	480
11	650
12	1300
13	570
14	1470
15	860
16	820
17	700
18	248